

Curso Técnico de Nível Médio em Inteligência Artificial -2026

Organizador de Rotas de Entrega para Barcos Regionais

GITHUB

Elizabeth Freitas
Matheus Anverez

7
C
E
T
A
M
:
:

CONTEXTO



Na Amazônia, muitas comunidades dependem exclusivamente do transporte fluvial para deslocamento de pessoas e mercadorias.

Um planejamento eficiente das rotas contribui para:

- Redução de custos operacionais;
- Melhor aproveitamento do tempo de viagem;
- Maior eficiência logística;
- Melhor atendimento às comunidades ribeirinhas.

A Artéria Logística da Amazônia



O Motor da Região

A logística fluvial é fundamental para o transporte de pessoas, mercadorias e suprimentos vitais na região amazônica.



Dependência Absoluta

Inúmeras comunidades ribeirinhas dependem exclusivamente da navegação pelos rios para deslocamento e abastecimento diário.



O Custo do Caos

O planejamento inadequado infla drasticamente o tempo de viagem, eleva o consumo de combustível e multiplica os custos operacionais.

Objetivo

- Cadastrar novos destinos fluviais;
- Exibir a rota completa de navegação;
- Calcular a distância total percorrida;
- Calcular o tempo total da viagem;
- Identificar o destino mais distante;
- Ordenar destinos por distância;
- Simular atrasos provocados por chuva.



Programação como Bússola Estratégica



A Arquitetura do Motor Computacional

Linguagem Base: Python 3

Estruturas de Dados

Listas []

Dicionários {}

Strings ""

Variáveis Numéricas
(int, float)

Estruturas de Controle

Condicionais
(if, elif, else)

Repetição
(while, for)

Modularização

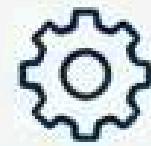
Escopo isolado
e reutilizável
através de
Funções (def)

Traduzindo o Mundo Real para a Memória do Sistema



Porto de Borba

Codificação
Digital



```
rota = [  
  {  
    "cidade": "Borba",  
    "distancia": 40,  
    "tempo": "3:00"  
  }  
]
```

"cidade" → O nome exato da comunidade ribeirinha.

"distancia" → Distância em quilômetros até a próxima parada.

"tempo" → Tempo estimado de deslocamento em formato (hh:mm).

Código

- Lista de dicionários contendo ascidades da rota fluvial.
- Cada registro armazena:
- Nome da cidade;
- Distância entre os trechos (km);
- Tempo médio de navegação.

Essas informações são utilizadas para consultas, ordenação e cálculo da distância e do tempo total da viagem.

```
# ORGANIZADOR DE ROTAS DE ENTREGA - RIO MADEIRA
# Manaus -> Nova Olinda -> Axinim -> Borba
# -> Novo Aripuanã -> Manicoré
# =====

rota = [
    {
        "cidade": "Nova Olinda do Norte",
        "distancia": 234, #km
        "tempo": "12:00 horas de viagem" #hora
    },
    {
        "cidade": "Axinim",
        "distancia": 37, #km
        "tempo": "2:30 horas de viagem" #hora
    },
    {
        "cidade": "Borba",
        "distancia": 49, #km
        "tempo": "3:00 horas de viagem" #hora
    },
    {
        "cidade": "Novo Aripuanã",
        "distancia": 119, #km
        "tempo": "9:00 horas de viagem" #hora
    },
    {
        "cidade": "Manicoré",
        "distancia": 128, #km
        "tempo": "11:00 horas de viagem" #hora
    }
]
```


Código

Consulta de Rotas

- Exibe a rota completa do ferry boat.
- Lista todos os destinos cadastrados.
- Apresenta distância e tempo de viagem de cada trecho.
- Facilita a visualização das informações armazenadas no sistema.

```
# -----  
# Listar rota completa  
# -----  
def listar_rota():  
  
    print("\n=== ROTA FLUVIAL DO RIO MADEIRA ===")  
  
    cidade_atual = "Manaus"  
  
    for destino in rota:  
  
        print(f"{cidade_atual} -> {destino['cidade']}")  
        print(f"Distância: {destino['distancia']} km")  
        print(f"Tempo: {destino['tempo']}")  
        print()  
  
        cidade_atual = destino["cidade"]  
  
# -----  
# Listar destinos  
# -----  
def listar_destinos():  
  
    print("\n=== DESTINOS CADASTRADOS ===")  
  
    for destino in rota:  
  
        print(  
            f"{destino['cidade']} - "  
            f"{destino['distancia']} km - "  
            f"{destino['tempo']}"  
        )
```

Bloco Lógico 1: Mapeando Novos Horizontes

```
1 def cadastrar_destino():
2     nome = input(...)
3     distancia = float(...)
4     tempo = input(...)
5
6     rota.append(...)
7
8
9
10
```



Função: Inserção de novas comunidades na rota principal.

Conceitos Aplicados: Coleta de entradas dinâmicas (input) armazenadas em Dicionários, que são imediatamente anexados a Listas (.append) para expandir o trajeto.

Código

Cadastro de Novos Destinos

- Permite adicionar uma nova cidade à rota fluvial.
- Solicita o nome da cidade, a distância e o tempo de viagem.
- Valida os dados informados para evitar erros de entrada.
- Armazena o novo destino na lista de rotas, atualizando o sistema automaticamente.

```
# Cadastro novo destino
# -----
def cadastrar_destino():

    nome = input("Nome da comunidade/cidade: ")

    while True:
        try:
            distancia = float(
                input("Distância até a próxima parada (km): ")
            )
            break # Sair do loop se a entrada for um número float válido
        except ValueError:
            print("Distância inválida. Por favor, insira um número (ex: 150 ou 2.5).")

    while True:
        tempo = input(
            "Tempo de viagem (hh:mm): "
        )
        try:
            # checar se o formato está correto.
            horas, minutos = tempo.split(":")
            int(horas)
            int(minutos)
            break # Sair do loop se o formato for válido
        except ValueError:
            print("Formato de tempo inválido. Por favor, use hh:mm.")

    rota.append({
        "cidade": nome,
        "distancia": distancia,
        "tempo": tempo
    })

    print("\nDestino cadastrado com sucesso!")
```

Otimização de Trajeto via Algoritmo de Ordenação

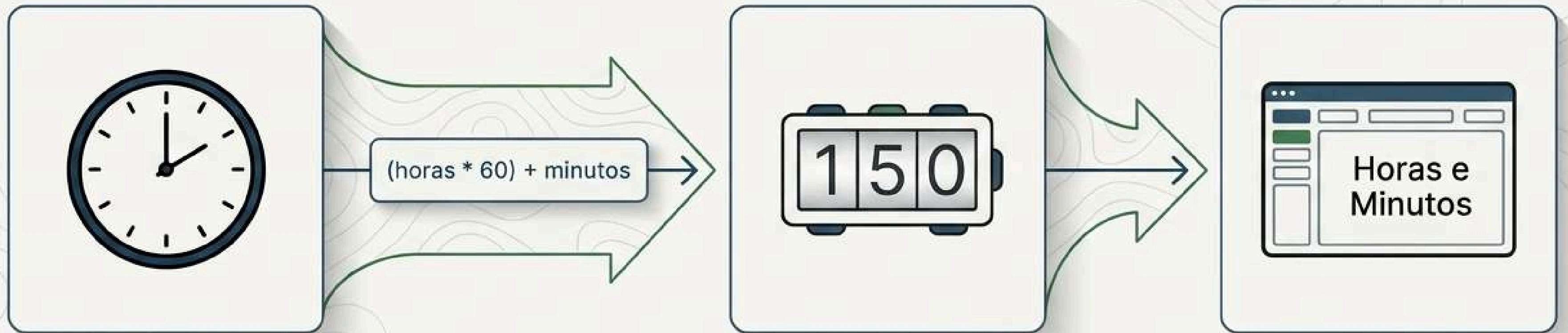
```
1 for i in range(n):
2     for j in range(n-i-1):
3         if rota[j]["distancia"] >
4             rota[j+1]["distancia"]:
5             # Realiza a troca (swap)
6
7
8
9
10
```



Método: Algoritmo Bubble Sort.

Objetivo Prático: Reorganizar a matriz de destinos para garantir que a leitura da rota ocorra do percurso mais curto para o mais longo de forma lógica e sequencial.

Padronização Matemática para Tempos de Viagem



Passo 1: Conversão de Formato

Extração computacional de horas e minutos do formato hh:mm.
(Exemplo: 2:30 converte-se em 150 minutos).

Passo 2: Agregação

Laço de repetição soma os minutos isolados de todos os trechos cadastrados na rota.

Passo 3: Reversão e Apresentação

O somatório final é reconvertido, entregando ao comandante do barco o tempo total exato da rota em Horas e Minutos legíveis.

Código

Cálculos da Rota

- Soma a distância de todos os trechos da viagem.
- Calcula o tempo total de navegação.
- Exibe os resultados de forma clara para o usuário.
- Auxilia no planejamento das entregas fluviais.

```
# -----  
def distancia_total():  
  
    total = 0  
  
    for destino in rota:  
  
        total += destino["distancia"]  
  
    print(  
        f"\nDistância total da rota: {total} km"  
    )  
  
    return total  
  
# -----  
# Tempo total  
# -----  
def tempo_total():  
  
    total_minutos = 0  
  
    for destino in rota:  
  
        total_minutos += converter_para_minutos(  
            destino["tempo"]  
        )  
  
    print(  
        "\nTempo total da rota:",  
        converter_para_horas(total_minutos)  
    )  
  
    return total_minutos
```

Modelagem Matemática da Imprevisibilidade Climática

$$\text{tempo_chuva} = \text{tempo_total} * 1.20$$

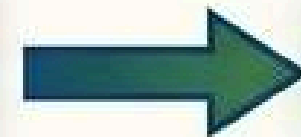
Cenário Padrão



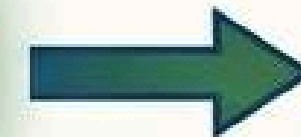
Navegação Padrão

Cálculo de tempo baseado na calha normal do rio.

Ex: 30h 00min



*** 1.20**



Cenário Adverso



Simulação de Chuva

Recálculo automático entregando o pior cenário possível.

Ex: 36h 00min

Código

Função `simular_chuva()`

- Calcula o tempo total da rota.
- Acrescenta 20% ao tempo para simular chuva.
- Compara o tempo normal com o tempo em condições climáticas adversas.
- Auxilia no planejamento logístico das entregas.

```
# Simulação de chuva (+20%)
# -----
def simular_chuva():

    total_minutos = 0

    for destino in rota:

        total_minutos += converter_para_minutos(
            destino["tempo"]
        )

    total_chuva = int(total_minutos * 1.20)

    print("\n== SIMULAÇÃO DE CHUVA ==")

    print(
        "Tempo normal:",
        converter_para_horas(total_minutos)
    )

    print(
        "Tempo com chuva:",
        converter_para_horas(total_chuva)
    )
```

Código

Menu do Sistema

- Centraliza o acesso às funcionalidades.
- Recebe a opção do usuário e executa a ação correspondente.
- Permite listar, cadastrar, ordenar e realizar cálculos sobre as rotas.
- Finaliza o programa de forma segura quando solicitado.

Código

```
# Menu Principal
# -----

while True:
    print("\n"=====)
    print(" ORGANIZADOR DE ROTAS FLUVIAIS ")
    print("=====")
    print("1 - Listar rota")
    print("2 - Listar destinos")
    print("3 - Cadastrar destino")
    print("4 - Ordenar por distância")
    print("5 - Calcular distância total")
    print("6 - Calcular tempo total")
    print("7 - Destino mais distante")
    print("8 - Simular atraso por chuva")
    print("0 - Sair")

    opcao = input("\nEscolha uma opção: ")

    if opcao == "1":
        listar_rota()
    elif opcao == "2":
        listar_destinos()
    elif opcao == "3":
        cadastrar_destino()
    elif opcao == "4":
        ordenar_por_distancia()
    elif opcao == "5":
        distancia_total()
    elif opcao == "6":
        tempo_total()
    elif opcao == "7":
        destino_mais_distante()
    elif opcao == "8":
        simular_chuva()
    elif opcao == "0":
        print("\nSistema encerrado.")
        break
    else:
        print("\nOpção inválida!")
```

Um Sistema Operacional Completo



Cadastro Dinâmico

Inserção de novos destinos em tempo real.



Visão Panorâmica

Listagem completa de toda a rota.



Motor de Otimização

Ordenação automática por distância percorrida.



Odômetro Central

Cálculo da distância total do trajeto.



Cronômetro Master

Cálculo do tempo total de navegação.



Alerta de Fronteira

Identificação imediata do destino mais distante.



Previsibilidade Operacional

Simulação de impacto por chuvas fortes.



Cronômetro

Cálculo do tempo de navegação.

Conclusão

O projeto demonstrou a aplicação prática dos conceitos de programação em um contexto regional amazônico.

Foram utilizados:

- Estruturas de dados;
- Funções;
- Laços de repetição;
- Condicionais;
- Algoritmos de ordenação.

O sistema permite organizar informações logísticas e simular situações que ocorrem em rotas fluviais reais.

A wide, calm river flows through a dense, lush green forest. The water is dark and reflects the surrounding trees and the overcast sky. The banks are covered in thick vegetation, including tall trees and hanging vines. In the distance, the river curves slightly to the right. The sky is filled with heavy, grey clouds. The overall mood is serene and natural.

Obrigado!!